S

Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology

Course Code
ECE 4124

Course Title
Digital Signal Processing Sessional

Experiment Date: July 21, 2025
Submission Date: August 2, 2025

Lab Report 2:
Study of Shifting, Folding & Scaling of Discrete Signals

Submitted to	**Submitted by**
Md. Faysal Ahamed	Md. Tajim An Noor
Lecturer	Roll: 2010025
Dept of ECE, Ruet	

Contents

Study of Shifting, Folding & Scaling of Discrete Signals

Theory

In digital signal processing, the manipulation of discrete-time signals through shifting, folding, and scaling is fundamental for analyzing and designing systems [1, 2, 3]. These operations allow engineers to modify signals for various applications, such as filtering, modulation, and system identification. Understanding these basic transformations is essential for interpreting how signals behave in both theoretical and practical contexts.

1. Shifting (Time Shifting):
Shifting a signal involves moving it forward or backward in time. For a discrete signal $x[n]$, the shifted signal is defined as:

$$y[n] = x[n - n_0] \tag{1}$$

where n_0 is the shift amount. If $n_0 > 0$, the signal is delayed; if $n_0 < 0$, it is advanced [1]. Time shifting is commonly used to synchronize signals, introduce delays, or align events in digital systems.

Example:
If $x[n] = \{1, 2, 3\}$ for $n = 0, 1, 2$, then $x[n - 1] = \{0, 1, 2, 3\}$ for $n = 0, 1, 2, 3$.

Applications:
Time shifting is crucial in echo generation, digital communication (for symbol alignment), and in implementing system responses where delays are inherent.

2. Folding (Time Reversal):
Folding reverses the signal in time. The folded signal is:

$$y[n] = x[-n] \tag{2}$$

This operation is useful for analyzing system symmetry, convolution, and for understanding the behavior of signals under time reversal [2]. Folding is also used in correlation analysis and in the study of even and odd signal components.

Example:
If $x[n] = \{1, 2, 3\}$ for $n = 0, 1, 2$, then $x[-n] = \{3, 2, 1\}$ for $n = -2, -1, 0$.

Applications:
Time reversal is essential in convolution operations, especially when calculating the output of linear time-invariant (LTI) systems, and in signal matching tasks.

3. Scaling (Time Scaling):
Scaling compresses or expands the signal in time. For integer a:

$$y[n] = x[an] \tag{3}$$

If $|a| > 1$, the signal is compressed; if $0 < |a| < 1$, it is expanded [3]. Time scaling changes the rate at which the signal evolves, which is important in applications such as sampling rate conversion and time-stretching in audio processing.

Example:
If $x[n] = \{1, 2, 3, 4\}$ for $n = 0, 1, 2, 3$, then $x[2n] = \{1, 3\}$ for $n = 0, 1$.

Applications:
Time scaling is used in speed adjustment of audio signals, resampling in digital communications, and in multi-rate signal processing systems.

Summary:
These operations—shifting, folding, and scaling—are essential for understanding signal behavior and system responses [1, 2, 3]. Shifting changes the timing, folding reverses the sequence, and scaling alters the rate at which the signal evolves. Mastery of these concepts is crucial for signal analysis, system design, and practical processing tasks. By applying these transformations, engineers can manipulate signals to meet specific requirements, analyze system properties, and develop robust digital signal processing solutions.

Used Tools

- Matlab
- LaTex

Code & Output

1. Discrete Sinusoidal Signal

```
% Parameters
n = -10:10;        % Discrete time index
A = 1;             % Amplitude
f = 0.05;          % Frequency (cycles/sample)
phi = 0;           % Phase

% Original sinusoidal signal
```

```
x = A * sin(2*pi*f*n + phi);

% Time shifting: x(n-3)
x_shift = A * sin(2*pi*f*(n-3) + phi);

% Folding: x(-n)
x_fold = A * sin(2*pi*f*(-n) + phi);

% Scaling: x(3n)
x_scale = A * sin(2*pi*f*(3*n) + phi);

% Plotting
figure;
subplot(2,2,1);
stem(n, x, 'filled');
title('Original: x[n]');
xlabel('n'); ylabel('x[n]');

subplot(2,2,2);
stem(n, x_shift, 'filled');
title('Time Shifted: x[n-3]');
xlabel('n'); ylabel('x[n-3]');

subplot(2,2,3);
stem(n, x_fold, 'filled');
title('Folded: x[-n]');
xlabel('n'); ylabel('x[-n]');

subplot(2,2,4);
stem(3*n, x_scale, 'filled');
title('Scaled: x[3n]');
xlabel('3n'); ylabel('x[3n]');
```

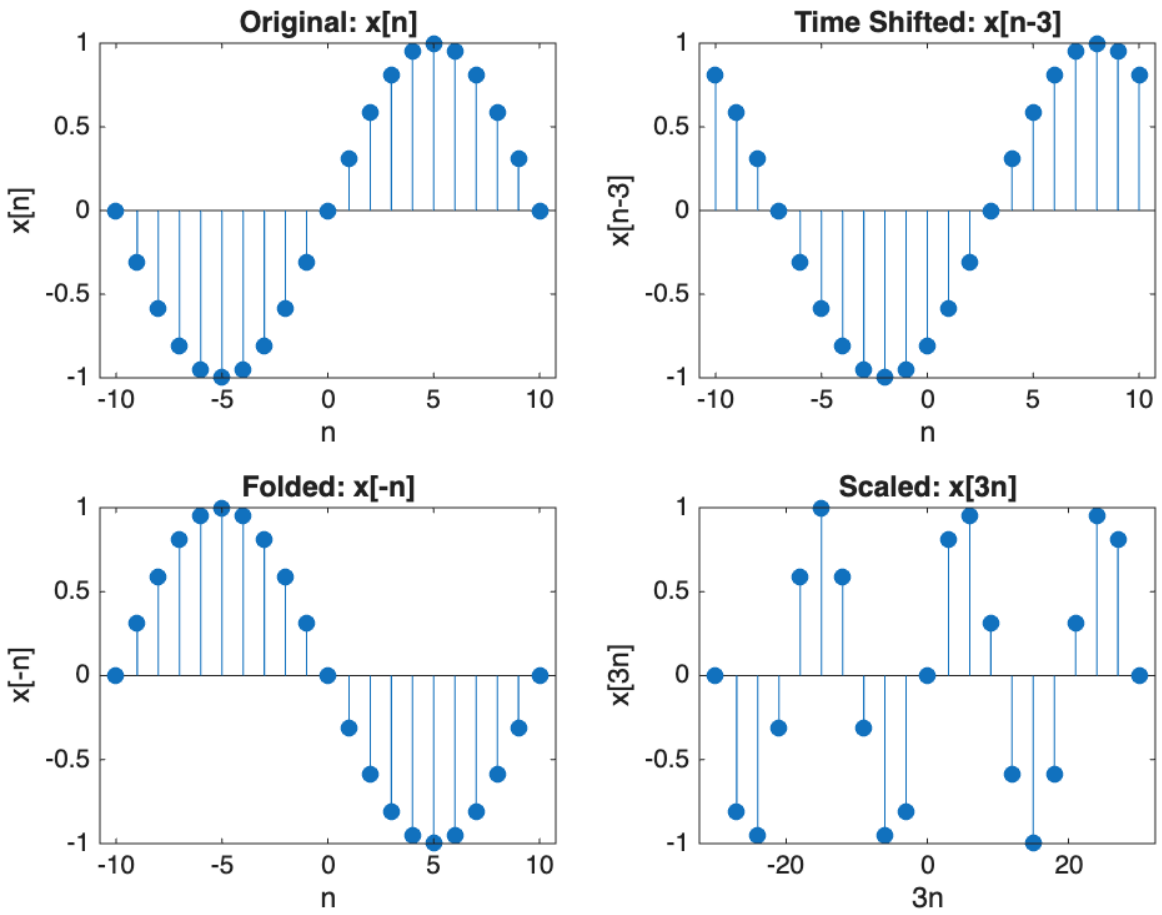

Figure 1: Discrete Sinusoidal Signal and its Shifting, Folding, Scaling

2. Discrete Cosine Signal

```
% Parameters
n = -10:10;         % Discrete time index
A = 1;              % Amplitude
f = 0.05;           % Frequency (cycles/sample)
phi = 0;            % Phase

% Original cosinusoidal signal
x = A * cos(2*pi*f*n + phi);

% Time shifting: x(n-3)
x_shift = A * cos(2*pi*f*(n-3) + phi);

% Folding: x(-n)
x_fold = A * cos(2*pi*f*(-n) + phi);

% Scaling: x(3n)
x_scale = A * cos(2*pi*f*(3*n) + phi);
```

```
18
19  % Plotting
20  figure;
21  subplot(2,2,1);
22  stem(n, x, 'filled');
23  title('Original: x[n]');
24  xlabel('n'); ylabel('x[n]');
25
26  subplot(2,2,2);
27  stem(n, x_shift, 'filled');
28  title('Time Shifted: x[n-3]');
29  xlabel('n'); ylabel('x[n-3]');
30
31  subplot(2,2,3);
32  stem(n, x_fold, 'filled');
33  title('Folded: x[-n]');
34  xlabel('n'); ylabel('x[-n]');
35
36  subplot(2,2,4);
37  stem(3*n, x_scale, 'filled');
38  title('Scaled: x[3n]');
39  xlabel('3n'); ylabel('x[3n]');
```

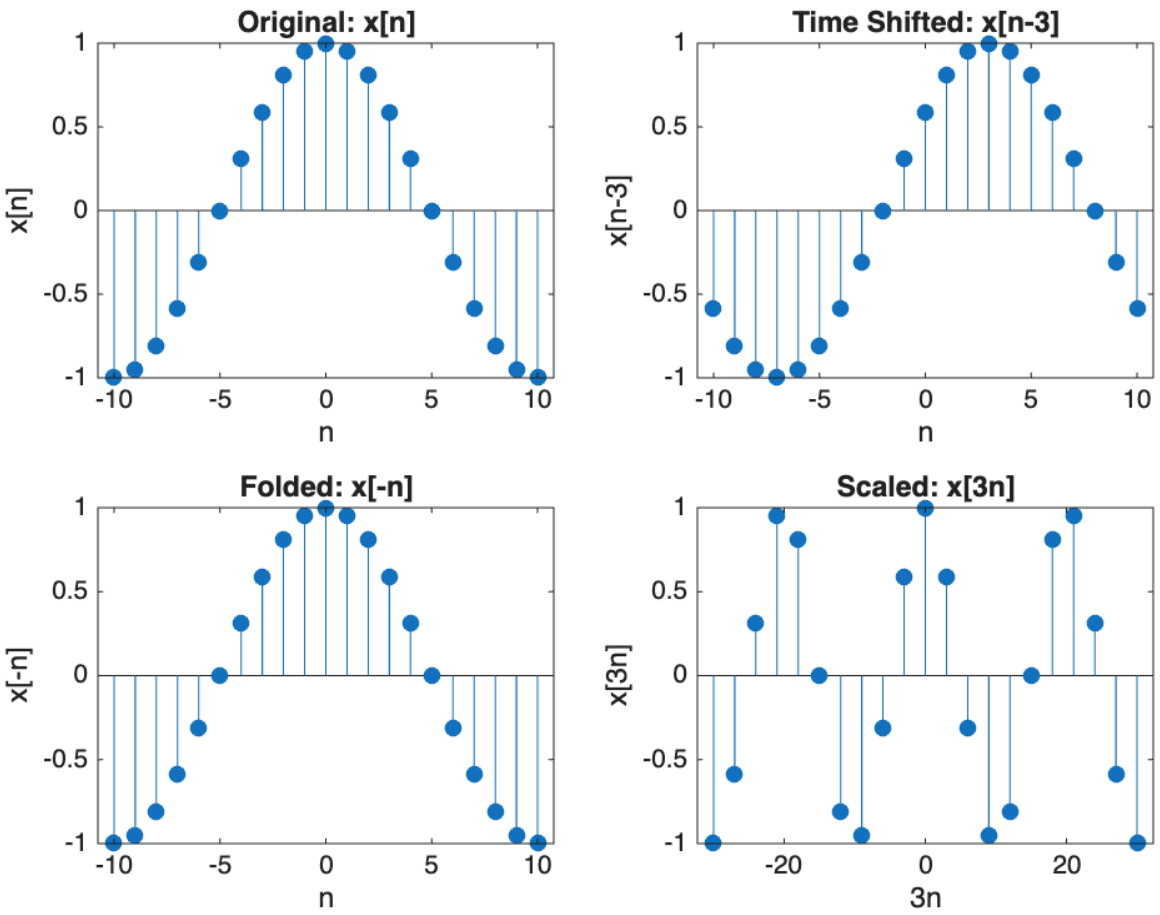

Figure 2: Discrete Cosine Signal and its Shifting, Folding, Scaling

3. Custom Signal $x(n) = (1, -3, 4, -5, 0, 2, 3, 5, -7)$

```
% Original signal x(n)
x = [1, -3, 4, -5, 0, 2, 3, 5, -7];
n = 0:length(x)-1; % n = 0 to 8

% 1. Time shifting: x(n-3)
shift_amount = 3;
n_shift = n - shift_amount;
x_shift = x;

% 2. Folding: x(-n)
n_fold = -n;
x_fold = flip(x);

% 3. Scaling: x(3n)
n_scale = 0:floor((length(x)-1)/3);
x_scale = x(3*n_scale + 1); % MATLAB uses 1-based indexing

```

```
% Plotting
figure;

subplot(4,1,1);
stem(n, x, 'filled');
title('Original Signal: x(n)');
xlabel('n');
ylabel('x(n)');

subplot(4,1,2);
stem(n_shift, x_shift, 'filled');
title('Time Shifting: x(n-3)');
xlabel('n');
ylabel('x(n-3)');

subplot(4,1,3);
stem(n_fold, x_fold, 'filled');
title('Folding: x(-n)');
xlabel('n');
ylabel('x(-n)');

subplot(4,1,4);
stem(n_scale, x_scale, 'filled');
title('Scaling: x(3n)');
xlabel('n');
ylabel('x(3n)');
```

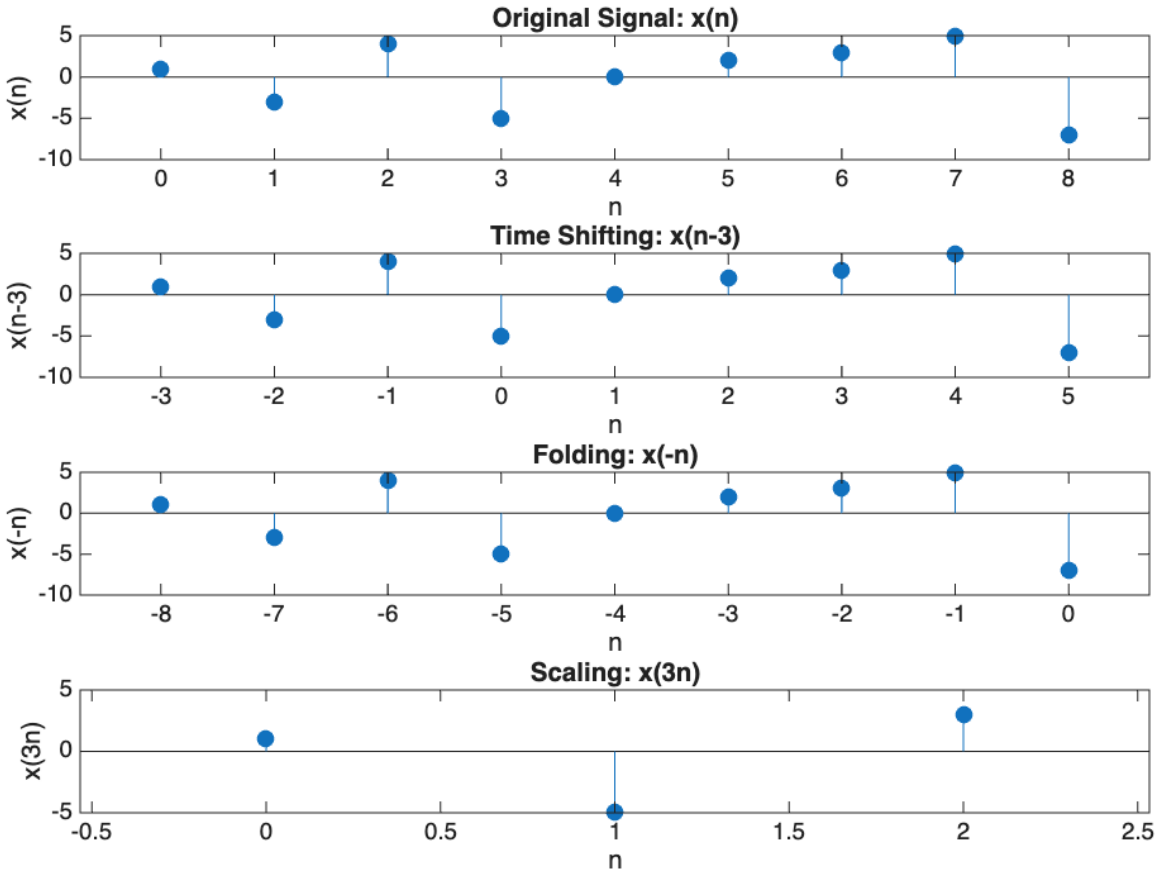

Figure 3: Custom Signal $x(n)$ and its Shifting, Folding, Scaling

Discussion

In this experiment, basic discrete-time signals—specifically the impulse, step, and ramp signals—were generated and transformed using MATLAB. The "cumsum" function [4] was utilized for computing the cumulative sum, enabling the conversion of a step signal into a ramp signal and an impulse signal into a step signal, thereby emulating discrete-time integration. Conversely, the "diff" function [5] was employed to perform discrete differentiation, allowing the derivation of the step signal from the ramp and the impulse signal from the step. The resulting plots from these operations were found to closely align with theoretical expectations, validating the correct implementation of these signal relationships. These hands-on conversions provided valuable insight into the mathematical links between the signals and reinforced understanding of their interdependence.

Conclusion

The experiment was conducted to illustrate the generation and transformation of fundamental discrete-time signals using essential MATLAB functions. Through ex-

amination of the characteristics and interconnections of the impulse, step, and ramp signals, practical understanding of their significance in digital signal processing was obtained. Functions such as `cumsum` and `diff` were employed to model discrete-time integration and differentiation, respectively, and theoretical principles were confirmed via graphical results.

References

[1] S. Salivahanan, A. Vallavaraj, and C. Gnanapriya, *Digital Signal Processing*, 3rd ed. McGraw-Hill Education, 2017.

[2] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, 2006.

[3] B. P. Lathi and R. Green, *Signals and Systems*, 2nd ed. Oxford University Press, 2014.

[4] The MathWorks, Inc., *cumsum: Cumulative sum of array elements*, 2024, mATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/ref/cumsum.html

[5] ——, *diff: Differences and approximate derivatives*, 2024, mATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/ref/diff.html